

Scripts Documentation

High5

June 28, 2025

Contents

1	merge_data.py	2
2	handle_nulls.py	2
3	add_cluster_info.py	5
4	add_cell_info_v2.py	7
5	exp_optuna_v2.py	9
6	do_experiment_v2.py	13
7	map_app.py	15
8	coverage_intersections.py	16

1 merge_data.py

Purpose: Merges multiple datasets into a single dataframe.

```
1 # Merging new dataset provided by teknofest
2 import polars as pl
3
4 dl_data = pl.read_excel("data/5g-field-data/5G_DL.xlsx", sheet_name="Series_Formatted_
    Data")
5 dl_test_data = pl.read_excel("data/5g-field-data/test-data/5G_DL.xlsx", sheet_name="
    Series_Formatted_Data")
6
7 dl_test_data = dl_test_data.with_columns((pl.col("Message")+ dl_data["Message"][-1] + 1)
    )
8
9 for col in dl_data.columns:
10     if col not in dl_test_data.columns:
11         dl_data = dl_data.drop(col)
12
13 for col in dl_test_data.columns:
14     if col not in dl_data.columns:
15         dl_test_data = dl_test_data.drop(col)
16
17 cat_cols = [
18     "NR_UE_PCI_0",
19     "NR_UE_Nbr_PCI_0",
20     "NR_UE_Nbr_PCI_1",
21     "NR_UE_Nbr_PCI_2",
22     "NR_UE_Nbr_PCI_3",
23     "NR_UE_RI_DL_0",
24     "NR_UE_Modulation_Avg_DL_0",
25     "NR_UE_MCS_DL_0",
26     "NR_UE_CCE_AggregationLev_0",
27     "NR_RRC_MsgType",
28     "NR_UE_RRCReEst_EndResult",
29     "NAS_5GS_MM_MessageType"
30 ]
31
32 merged_data = pl.concat([dl_data,dl_test_data],how='vertical_relaxed')
33
34 merged_data = merged_data.drop("Technology_Mode")
35
36 for col in merged_data.columns:
37     if col == "Time":
38         continue
39     elif "_PCI_" in col:
40         merged_data = merged_data.with_columns(
41             pl.col(col).cast(pl.Float64, strict=True)
42         )
43     elif (merged_data[col].dtype == pl.String) and col not in cat_cols:
44         merged_data = merged_data.with_columns(
45             pl.col(col).cast(pl.Float64, strict=True)
46         )
47
48 merged_data.write_parquet("data/parquet/dl.parquet")+"

```

2 handle_nulls.py

Purpose: Imputation of null values.

```
1 import polars as pl
2 from calculations_util.distance import pathloss_to_distance
3
4 cell_data = pl.read_parquet("data/parquet/cell_info.parquet")
5 df = pl.read_parquet("data/parquet/dl.parquet").sort("Message")
6
7
8 df_flags = (
9     df.select(

```

```

10         "Time",
11         "NR_UE_RRCHOAttempt",
12         "NR_UE_RRCHOOK",
13         "NR_UE_RACH_OK",
14         "NR_UE_RACH_Attempt",
15     )
16     .group_by("Time")
17     .agg(
18         [
19             (pl.col("NR_UE_RACH_Attempt") == 1)
20             .any()
21             .cast(pl.Int8)
22             .alias("rach_attempt"),
23             (pl.col("NR_UE_RACH_OK") == 1).any().cast(pl.Int8).alias("rach_ok"),
24             (pl.col("NR_UE_RRCHOAttempt") == 1).any().cast(pl.Int8).alias("ho_attempt"),
25             (pl.col("NR_UE_RRCHOOK") == 1).any().cast(pl.Int8).alias("ho_ok"),
26         ]
27     )
28 )
29
30 fill_cols = [
31     "NR_UE_Timing_Advance",
32     "NR_UE_Pathloss_DL_0",
33     "NR_UE_Throughput_PDCH_DL",
34     "App_Throughput_DL",
35     "NR_UE_NACK_Rate_DL_0",
36     "NR_UE_Ack_As_Nack_DL_0",
37     "NR_UE_MCS_DL_0",
38     "NR_UE_RB_Num_DL_0",
39     "NR_UE_Modulation_Avg_DL_0",
40     "NR_UE_RI_DL_0",
41     "NR_UE_BLER_DL_0",
42     "NR_UE_CCE_AggregationLev_0",
43     "NR_UE_Power_Tx_PUSCH_0",
44     "NR_UE_Power_Tx_PRACH_0",
45     "NR_UE_NACK_Rate_UL_0",
46 ]
47
48 df = df.with_columns([
49     pl.col(col).forward_fill().backward_fill().over("Time").alias(col)
50     for col in fill_cols
51 ])
52
53 df = df.filter(
54     pl.col("NR_RRC_MsgType").is_null()
55 ).drop(
56     [
57         "NR_UE_RRCHOAttempt",
58         "NR_UE_RRCHOOK",
59         "NR_UE_RACH_OK",
60         "NR_UE_RACH_Attempt",
61         "NR_RRC_MsgType",
62     ]
63 )
64
65 group_cols = [
66     "Longitude",
67     "Latitude",
68     "Time",
69     "NR_UE_PCI_0",
70     "NR_UE_RSRP_0",
71     "NR_UE_RSRQ_0",
72     "NR_UE_SINR_0",
73     "NR_UE_Nbr_PCI_0",
74     "NR_UE_Nbr_PCI_1",
75     "NR_UE_Nbr_PCI_2",
76     "NR_UE_Nbr_PCI_3",
77     "NR_UE_Nbr_RSRP_0",
78     "NR_UE_Nbr_RSRP_1",
79     "NR_UE_Nbr_RSRP_2",
80     "NR_UE_Nbr_RSRP_3",
81     "NR_UE_Nbr_RSRQ_0",
82     "NR_UE_Nbr_RSRQ_1",

```

```

83     "NR_UE_Nbr_RSRQ_2",
84     "NR_UE_Nbr_RSRQ_3",
85 ]
86
87 df = (
88     df.groupby(group_cols)
89     .agg(
90         pl.col("Message").max(), *[pl.col(col).first().alias(col) for col in fill_cols]
91     )
92     .join(df_flags, on="Time")
93     .drop("Time")
94     .filter(pl.col("NR_UE_PCI_0").is_in(list(cell_data["PCI"])),
95            pl.col("NR_UE_PCI_0").is_not_null(),
96            pl.col("NR_UE_RSRP_0").is_not_null())
97 )
98
99 # replacing with bit/symbol
100
101 df = df.with_columns(
102     [
103         pl.when(pl.col("NR_UE_Modulation_Avg_DL_0") == "QPSK")
104         .then(4)
105         .when(pl.col("NR_UE_Modulation_Avg_DL_0") == "16QAM")
106         .then(16)
107         .when(pl.col("NR_UE_Modulation_Avg_DL_0") == "64QAM")
108         .then(64)
109         .when(pl.col("NR_UE_Modulation_Avg_DL_0") == "256QAM")
110         .then(256)
111         .otherwise(None)
112         .alias("modulation_avg_encoded"),
113     ]
114 )
115
116 df = df.with_columns(
117     [pl.col("modulation_avg_encoded").log(base=2).alias("modulation_avg_log2_bits")]
118 )
119
120 df = df.with_columns(
121     [
122         pl.when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_1")
123         .then(1)
124         .when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_2")
125         .then(2)
126         .when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_4")
127         .then(4)
128         .when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_8")
129         .then(8)
130         .when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_16")
131         .then(16)
132         .otherwise(None)
133         .alias("cce_level_encoded")
134     ]
135 )
136
137 df = df.with_columns(
138     [
139         pl.col("NR_UE_MCS_DL_0")
140         .str.extract(r"~(\d+)", group_index=1) # extracting mcs index at the beggining
141         .cast(pl.Int64)
142         .alias("mcs_index")
143     ]
144 )
145
146 df = df.with_columns(
147     pathloss_to_distance(pl.col("NR_UE_Pathloss_DL_0")).alias("pl_distance")
148 )
149
150 df = df.with_columns(
151     [
152         (pl.col("NR_UE_Pathloss_DL_0") / pl.col("pl_distance")).alias("pl_per_meter"),
153         pl.col("pl_distance").log1p().alias("log1p_pl_distance"),
154         (pl.col("App_Throughput_DL") / pl.col("NR_UE_RB_Num_DL_0")).alias(
155             "spectral_efficiency"
156         )
157     ]
158 )

```

```

156         ),
157         (pl.col("modulation_avg_log2_bits") / (pl.col("NR_UE_RB_Num_DL_0"))).alias(
158             "modulation_efficiency"
159         ),
160         (pl.col("NR_UE_Throughput_PDCP_DL") / pl.col("pl_distance")).alias(
161             "throughput_per_meter"
162         ),
163         (pl.col("NR_UE_SINR_0") / (pl.col("pl_distance"))).alias("sinr_per_meter"),
164         (pl.col("NR_UE_SINR_0") / (pl.col("NR_UE_RSRP_0"))).alias("sinr_over_rsrp"),
165         (pl.col("NR_UE_RSRP_0") - pl.col("NR_UE_Nbr_RSRP_0")).alias("rsrp_delta"),
166         (pl.col("NR_UE_RSRQ_0") - pl.col("NR_UE_Nbr_RSRQ_0")).alias("rsrq_delta"),
167         (pl.col("NR_UE_RSRP_0") / pl.col("NR_UE_Pathloss_DL_0")).alias(
168             "signal_loss_ratio"
169         ),
170     ]
171 )
172
173 df.write_parquet("data/parquet/processed/dl.parquet")

```

3 add_cluster_info.py

Purpose: Adds cluster information to the dataset for segmentation purposes.

```

1
2 # create folds with agglomerative clustering and save folds as a pickle file.
3 import os
4 import pickle
5 import polars as pl
6 import geopandas as gpd
7 from sklearn.metrics.pairwise import haversine_distances
8 from sklearn.cluster import AgglomerativeClustering
9 from pyproj import Geod
10
11 geod = Geod(ellps="WGS84")
12
13 df = pl.read_parquet("data/parquet/processed/dl.parquet")
14 df_clean = df.drop_nulls(subset=["Longitude", "Latitude"]).select(
15     pl.col("Longitude").radians(), pl.col("Latitude").radians(), "Message"
16 )
17
18 n_clusters = 10
19 X = df_clean[["Latitude", "Longitude"]].to_numpy()
20
21 agg = AgglomerativeClustering(
22     n_clusters=n_clusters, metric=haversine_distances, linkage="complete"
23 )
24 agg.fit_predict(X)
25
26 complete = df.join(
27     df_clean.with_columns(pl.Series(name="cluster", values=agg.fit_predict(X))).select(
28         "cluster", "Message"
29     ),
30     on="Message",
31     how="left",
32 )
33
34 complete.write_parquet("data/parquet/processed/dl.with_cluster.parquet")
35
36 cluster_centers = {}
37
38 for cluster in range(n_clusters):
39     df_filtered = complete.select("Longitude", "Latitude", "cluster").filter(
40         (pl.col("cluster") == cluster)
41     )
42
43     geometry = gpd.points_from_xy(
44         df_filtered["Longitude"], df_filtered["Latitude"], crs="EPSG:4326"
45     )
46     gdf = gpd.GeoDataFrame(geometry=geometry)
47     gdf = gdf.to_crs(

```

```

48     "EPSG:32635"
49 ) # projecting to local UTM zone for accurate centroid calculation.
50 centroid = gdf.union_all().centroid
51 centroid_gdf = gpd.GeoDataFrame(geometry=[centroid], crs="EPSG:32635").to_crs(
52     "EPSG:4326"
53 ) # converting centroid back
54 centroid = centroid_gdf.geometry.x.iloc[0], centroid_gdf.geometry.y.iloc[0]
55 cluster_centers[cluster] = centroid
56
57 # print(f"clusters and cluster centroids:\n{cluster_centers}")
58
59 distance_dict = {}
60 distance_threshold = 700
61
62 for i in range(len(cluster_centers)):
63     lon1, lat1 = cluster_centers[i]
64     for j in range(i + 1, len(cluster_centers)):
65         lon2, lat2 = cluster_centers[j]
66         _, _, distance = geod.inv(lon1, lat1, lon2, lat2)
67         if distance_threshold < distance:
68             distance_dict[(i, j)] = distance
69
70 # print(f"distance between fold centers:\n{distance_dict}")
71
72 folds = {}
73
74 for pairs in distance_dict:
75     if pairs[0] not in folds:
76         folds[pairs[0]] = [pairs[1]]
77     else:
78         folds[pairs[0]].append(pairs[1])
79     if pairs[1] not in folds:
80         folds[pairs[1]] = [pairs[0]]
81     else:
82         folds[pairs[1]].append(pairs[0])
83
84
85 # deleting folds where test fold size > train fold size
86
87 del_folds = False
88 if del_folds:
89     for fold in folds.copy():
90         test_cluster = fold
91         train_clusters = folds[fold]
92
93         test_size = len(complete.filter(pl.col("cluster") == test_cluster))
94         train_size = len(complete.filter(pl.col("cluster").is_in(train_clusters)))
95
96         if test_size > train_size:
97             del folds[fold]
98
99 os.makedirs("data/folds", exist_ok=True)
100
101 with open("data/folds/folds.pkl", "wb") as f:
102     pickle.dump(folds, f)
103
104 # print(f"folds:\n{folds}")
105
106 # plot clusters if you want to check
107 """
108 df = pl.read_parquet("data/parquet/processed/dl.with_cluster.parquet")
109
110 plt.figure(figsize=(10, 6))
111 scatter = plt.scatter(
112     df["Longitude"],
113     df["Latitude"],
114     c=df["cluster"],
115     cmap="tab10",
116     s=10,
117 )
118
119 for cluster_id, (lon, lat) in cluster_centers.items():
120     plt.scatter(lon, lat, c="red", marker="o", s=100)

```

```

121     plt.text(lon, lat, str(cluster_id), color="black", fontsize=15, ha="center")
122
123 plt.xlabel("Longitude")
124 plt.ylabel("Latitude")
125 plt.grid(True)
126 plt.show()
127 """

```

4 add_cell_info_v2.py

Purpose: Adds base station (cell) information to enrich the dataset.

```

1  import polars as pl
2  import geopandas as gpd
3  from pyproj import Geod
4
5  geod = Geod(ellps="WGS84")
6
7  cell_data = pl.read_parquet("data/parquet/cell_info.parquet")
8  df = pl.read_parquet("data/parquet/processed/dl.with_cluster.parquet")
9  coverages = gpd.read_parquet("data/parquet/coverages/coverages.parquet")
10 intersections_2 = gpd.read_parquet(
11     "data/parquet/coverages/coverages_intersect_2.parquet"
12 )
13
14 coverages = pl.DataFrame(
15     {
16         "centroid_longitude": coverages.clipped_coverage_centroid.x,
17         "centroid_latitude": coverages.clipped_coverage_centroid.y,
18         "PCI": coverages["PCI"],
19     }
20 )
21
22 intersections_2 = pl.DataFrame(
23     {
24         "sorted_cell_pairs": intersections_2["sorted_cell_pairs"],
25         "intersection_2_convex_hull": intersections_2["convex_hull"],
26         "intersection_2_area_m2": intersections_2["area_m2"],
27         "intersection_2_centroid_lon": intersections_2["centroid"].x,
28         "intersection_2_centroid_lat": intersections_2["centroid"].y,
29     }
30 )
31
32 pci_cols = [col for col in df.columns if "PCI" in col]
33
34 for i, pci_col in enumerate(pci_cols):
35     df = df.join(
36         cell_data.select(
37             pl.col("Longitude"),
38             pl.col("Latitude"),
39             pl.col("Azimuth_[" + pci_col + "]").radians().sin().alias(f"Azimuth_nn{i}_cell_sin"),
40             pl.col("Azimuth_[" + pci_col + "]").radians().cos().alias(f"Azimuth_nn{i}_cell_cos"),
41             pl.when(pl.col("MIMO") == "64T64R").then(64)
42             .when(pl.col("MIMO") == "32T32R").then(32)
43             .otherwise(None)
44             .alias(f"mimo_nn{i}_cell"),
45             "PCI",
46         ),
47         left_on=pci_col,
48         right_on="PCI",
49         how="left",
50         suffix=f"_nn{i}_cell",
51     )
52
53 # joining connected cell's azimuth in degrees, which is going to be used for geod
54 # fwd function below.
55 if i == 0:
56     df = df.join(
57         cell_data.select(
58             pl.col("Azimuth_[" + pci_col + "]").alias("Azimuth_nn0_cell"),

```

```

58         "PCI",
59     ),
60     left_on=pci_col,
61     right_on="PCI",
62     how="left",
63 )
64
65 df = df.join(
66     coverages.select(
67         pl.col("centroid_longitude").alias(f"centroid_lon_nn{i}_cell_coverage"),
68         pl.col("centroid_latitude").alias(f"centroid_lat_nn{i}_cell_coverage"),
69         "PCI",
70     ),
71     left_on=pci_col,
72     right_on="PCI",
73     how="left",
74 )
75 df = (
76     df.with_columns(
77         pl.concat_list([pl.col("NR_UE_PCI_0"), pl.col("NR_UE_Nbr_PCI_0")])
78         .list.sort()
79         .alias("sorted_cell_pairs")
80     )
81     .join(intersections_2, how="left", on="sorted_cell_pairs")
82     .drop("sorted_cell_pairs")
83 )
84
85 ue_lon, ue_lat, _ = geod.fwd(
86     lons=df["Longitude_nn0_cell"],
87     lats=df["Latitude_nn0_cell"],
88     az=df["Azimuth_nn0_cell"],
89     #dist=df["distance"], old column name
90     dist=df["pl_distance"],
91 )
92
93 df = df.with_columns(
94     [pl.Series("pl_predicted_longitude", ue_lon), pl.Series("pl_predicted_latitude",
95         ue_lat)
96 ]
97 )
98
99 """
100 old df column names
101
102 df = df.with_columns([
103     (pl.col("spectral_efficiency") / pl.col("mimo_nn0_cell")).alias("
104         efficiency_per_antenna"),
105     (pl.col("m_NR_UE_SINR_0") / pl.col("mimo_nn0_cell")).alias("sinr_per_antenna"),
106     (pl.col("mimo_nn0_cell") / (pl.col("distance") + 1)).alias("mimo_distance_ratio"),
107     (pl.col("mimo_nn0_cell") * pl.col("m_App_Throughput_DL")).alias("
108         mimo_throughput_potential"),
109     (pl.col("signal_loss_ratio") * pl.col("mimo_nn0_cell")).alias("scaled_loss_ratio"),
110     (pl.col("Azimuth_nn0_cell_cos") * -pl.col("m_NR_UE_RSRP_0")).alias("
111         directional_rsrp_nn0_lon"),
112     (pl.col("Azimuth_nn0_cell_sin") * -pl.col("m_NR_UE_RSRP_0")).alias("
113         directional_rsrp_nn0_lat"),
114     (pl.col("Azimuth_nn1_cell_cos") * -pl.col("m_NR_UE_Nbr_RSRP_1")).alias("
115         directional_rsrp_nn1_lon"),
116     (pl.col("Azimuth_nn1_cell_sin") * -pl.col("m_NR_UE_Nbr_RSRP_1")).alias("
117         directional_rsrp_nn1_lat"),
118     (pl.col("Azimuth_nn2_cell_cos") * -pl.col("m_NR_UE_Nbr_RSRP_2")).alias("
119         directional_rsrp_nn2_lon"),
120     (pl.col("Azimuth_nn2_cell_sin") * -pl.col("m_NR_UE_Nbr_RSRP_2")).alias("
121         directional_rsrp_nn2_lat"),
122     (pl.col("Azimuth_nn3_cell_cos") * -pl.col("m_NR_UE_Nbr_RSRP_3")).alias("
123         directional_rsrp_nn3_lon"),
124     (pl.col("Azimuth_nn3_cell_sin") * -pl.col("m_NR_UE_Nbr_RSRP_3")).alias("
125         directional_rsrp_nn3_lat"),
126 ])
127 """
128
129 df = df.with_columns([

```



```

120     (pl.col("spectral_efficiency") / pl.col("mimo_nn0_cell")).alias("
121         efficiency_per_antenna"),
122     (pl.col("NR_UE_SINR_0") / pl.col("mimo_nn0_cell")).alias("sinr_per_antenna"),
123     (pl.col("mimo_nn0_cell") / (pl.col("pl_distance") + 1)).alias("mimo_distance_ratio")
124     ,
125     (pl.col("mimo_nn0_cell") * pl.col("App_Throughput_DL")).alias("
126         mimo_throughput_potential"),
127     (pl.col("signal_loss_ratio") * pl.col("mimo_nn0_cell")).alias("scaled_loss_ratio"),
128     (pl.col("Azimuth_nn0_cell_cos") * -pl.col("NR_UE_RSRP_0")).alias("
129         directional_rsrp_nn0_lon"),
130     (pl.col("Azimuth_nn0_cell_sin") * -pl.col("NR_UE_RSRP_0")).alias("
131         directional_rsrp_nn0_lat"),
132     (pl.col("Azimuth_nn1_cell_cos") * -pl.col("NR_UE_Nbr_RSRP_1")).alias("
133         directional_rsrp_nn1_lon"),
134     (pl.col("Azimuth_nn1_cell_sin") * -pl.col("NR_UE_Nbr_RSRP_1")).alias("
135         directional_rsrp_nn1_lat"),
136     (pl.col("Azimuth_nn2_cell_cos") * -pl.col("NR_UE_Nbr_RSRP_2")).alias("
137         directional_rsrp_nn2_lon"),
138     (pl.col("Azimuth_nn2_cell_sin") * -pl.col("NR_UE_Nbr_RSRP_2")).alias("
139         directional_rsrp_nn2_lat"),
140     (pl.col("Azimuth_nn3_cell_cos") * -pl.col("NR_UE_Nbr_RSRP_3")).alias("
141         directional_rsrp_nn3_lon"),
142     (pl.col("Azimuth_nn3_cell_sin") * -pl.col("NR_UE_Nbr_RSRP_3")).alias("
143         directional_rsrp_nn3_lat"),
144 ]
145 df.write_parquet("data/parquet/processed/dl.with_cluster_cell_info.parquet")

```

5 exp_optuna_v2.py

Purpose: Runs experiments using Optuna for hyperparameter optimization.

```

1  import os
2  from pathlib import Path
3  import polars as pl
4  import pickle
5  from sklearn.ensemble import StackingRegressor
6  from sklearn.multioutput import MultiOutputRegressor
7  from lightgbm import LGBMRegressor
8  from sklearn.preprocessing import PolynomialFeatures
9  from sklearn.ensemble import RandomForestRegressor
10 from sklearn.pipeline import make_pipeline
11 from sklearn.model_selection import cross_val_score
12 from sklearn.metrics import make_scorer
13 import optuna
14 import webbrowser
15 from datetime import datetime
16 from xgboost import XGBRegressor
17 from cell_util.ml import CustomCV
18 from calculations_util.distance import haversine_np
19
20 df = (
21     pl.read_parquet("data/parquet/processed/dl.with_cluster_cell_info.parquet")
22     .drop_nulls(["Longitude", "Latitude"])
23     .select(
24         "pl_distance",
25         "log1p_pl_distance",
26         "pl_per_meter",
27         "pl_predicted_longitude",
28         "pl_predicted_latitude",
29         "throughput_per_meter",
30         "spectral_efficiency",
31         "sinr_per_meter",
32         "sinr_over_rsrp",
33         "signal_loss_ratio",
34         "rsrq_delta",
35         "rsrp_delta",
36         "scaled_loss_ratio",
37         "mimo_throughput_potential",
38     )

```

```

39     "mimo_distance_ratio",
40     "sinr_per_antenna",
41     "efficiency_per_antenna",
42     "modulation_efficiency",
43     "mimo_nn0_cell",
44     "mimo_nn1_cell",
45     "mimo_nn2_cell",
46     "mimo_nn3_cell",
47     "Longitude",
48     "Latitude",
49     "NR_UE_Pathloss_DL_0",
50     "Longitude_nn0_cell",
51     "Latitude_nn0_cell",
52     "Longitude_nn1_cell",
53     "Latitude_nn1_cell",
54     "Longitude_nn2_cell",
55     "Latitude_nn2_cell",
56     "Longitude_nn3_cell",
57     "Latitude_nn3_cell",
58     "Azimuth_nn0_cell_cos",
59     "Azimuth_nn0_cell_sin",
60     "Azimuth_nn1_cell_cos",
61     "Azimuth_nn1_cell_sin",
62     "Azimuth_nn2_cell_cos",
63     "Azimuth_nn2_cell_sin",
64     "Azimuth_nn3_cell_cos",
65     "Azimuth_nn3_cell_sin",
66     "directional_rsrp_nn0_lon",
67     "directional_rsrp_nn0_lat",
68     "directional_rsrp_nn1_lon",
69     "directional_rsrp_nn1_lat",
70     "directional_rsrp_nn2_lon",
71     "directional_rsrp_nn2_lat",
72     "directional_rsrp_nn3_lon",
73     "directional_rsrp_nn3_lat",
74     "cluster",
75     "NR_UE_RSRP_0",
76     "NR_UE_Nbr_RSRP_0",
77     "NR_UE_Nbr_RSRP_1",
78     "NR_UE_Nbr_RSRP_2",
79     "NR_UE_Nbr_RSRP_3",
80     "NR_UE_SINR_0",
81     "NR_UE_RB_Num_DL_0",
82     "NR_UE_RSRQ_0",
83     "NR_UE_Nbr_RSRQ_0",
84     "NR_UE_Nbr_RSRQ_1",
85     "NR_UE_Nbr_RSRQ_2",
86     "NR_UE_Nbr_RSRQ_3",
87     "NR_UE_BLER_DL_0",
88     "NR_UE_Timing_Advance",
89     "rach_attempt",
90     "rach_ok",
91     "ho_attempt",
92     "ho_ok",
93     "App_Throughput_DL",
94     "NR_UE_Throughput_PDCP_DL",
95     "NR_UE_Ack_As_Nack_DL_0",
96     "NR_UE_NACK_Rate_DL_0",
97     "NR_UE_Power_Tx_PUSCH_0",
98     "NR_UE_Power_Tx_PRACH_0",
99     "NR_UE_NACK_Rate_UL_0",
100    "modulation_avg_encoded",
101    "modulation_avg_log2_bits",
102    "cce_level_encoded",
103    "mcs_index",
104    "centroid_lon_nn0_cell_coverage",
105    "centroid_lat_nn0_cell_coverage",
106    "centroid_lon_nn1_cell_coverage",
107    "centroid_lat_nn1_cell_coverage",
108    "centroid_lon_nn2_cell_coverage",
109    "centroid_lat_nn2_cell_coverage",
110    "centroid_lon_nn3_cell_coverage",
111    "centroid_lat_nn3_cell_coverage",

```

```

112         "intersection_2_centroid_lon",
113         "intersection_2_centroid_lat",
114     )
115     .to_pandas()
116 )
117
118 with open("data/folds/folds.pkl", "rb") as f:
119     folds = pickle.load(f)
120
121
122 y = df[["Latitude", "Longitude"]].to_numpy()
123
124 X = df.drop(["Latitude", "Longitude", "cluster"], axis=1).to_numpy()
125
126 scorer = make_scorer(score_func=haversine_np, greater_is_better=False)
127
128
129 def objective(trial):
130     n_estimators = trial.suggest_int("n_estimators", 100, 700)
131     learning_rate = trial.suggest_float("learning_rate", 1e-5, 0.01)
132     max_depth = trial.suggest_int("max_depth", 3, 20)
133     # gamma = trial.suggest_float("gamma", 0.0, 1e-6) # trials may stuck at score
134     # -850.9125320592835 when upper bound = 1-e2 or higher
135     reg_alpha = trial.suggest_float("reg_alpha", 1e-4, 0.20)
136     reg_lambda = trial.suggest_float("reg_lambda", 0.0, 0.15)
137     subsample = trial.suggest_float("subsample", 0.5, 1.0)
138     colsample_bytree = trial.suggest_float("colsample_bytree", 0.5, 1.0)
139     # colsample_bylevel = trial.suggest_float("colsample_bylevel", 0.5, 1.0)
140     # colsample_bynode = trial.suggest_float("colsample_bynode", 0.5, 1.0)
141     min_child_weight = trial.suggest_int("min_child_weight", 1, 60)
142     grow_policy = trial.suggest_categorical("grow_policy", ["depthwise", "lossguide"])
143
144     if grow_policy == "lossguide":
145         max_leaves, max_depth = trial.suggest_int("max_leaves", 16, 512), 0
146     else:
147         max_leaves = 0
148
149     xgb = XGBRegressor(
150         n_estimators=n_estimators,
151         learning_rate=learning_rate,
152         max_depth=max_depth,
153         max_leaves=max_leaves,
154         reg_alpha=reg_alpha,
155         reg_lambda=reg_lambda,
156         subsample=subsample,
157         colsample_bytree=colsample_bytree,
158         min_child_weight=min_child_weight,
159         grow_policy=grow_policy,
160         sampling_method="uniform",
161         tree_method="hist",
162         n_jobs=-1,
163         objective="reg:squarederror",
164         seed=23,
165     )
166
167     """
168     lgbm_n_estimators = trial.suggest_int("lgbm_n_estimators", 100, 600)
169     lgbm_learning_rate = trial.suggest_float("lgbm_learning_rate", 1e-3, 0.01)
170     lgbm_max_depth = trial.suggest_int("lgbm_max_depth", 3, 12)
171     lgbm_reg_alpha = (trial.suggest_float("lgbm_reg_alpha", 0.0, 0.15),)
172     lgbm_reg_lambda = (trial.suggest_float("lgbm_reg_lambda", 0.0, 0.15),)
173     lgbm_importance_type = (
174         trial.suggest_categorical("lgbm_importance_type", ["split", "gain"]),
175     )
176     lgbm_subsample = trial.suggest_float("lgbm_subsample", 0.8, 1.0)
177
178     lgbm = LGBMRegressor(
179         n_estimators=lgbm_n_estimators,
180         learning_rate=lgbm_learning_rate,
181         max_depth=lgbm_max_depth,
182         reg_alpha=lgbm_reg_alpha,
183         reg_lambda=lgbm_reg_lambda,
184         subsample=lgbm_subsample,

```

```

184         importance_type=lgbm_importance_type,
185         objective="regression",
186         n_jobs=-1,
187         random_state=23,
188         verbose=-1,
189     )
190     """
191
192     #degree = trial.suggest_int("poly_degree", 1, 4)
193     """
194     meta_learner = make_pipeline(
195         PolynomialFeatures(degree=degree, include_bias=False),
196         RandomForestRegressor(n_estimators=100, random_state=23, n_jobs=-1),
197     )
198
199     stacking = StackingRegressor(
200         estimators=[("xgb", xgb)],
201         final_estimator=meta_learner,
202         n_jobs=-1,
203     )
204     """
205     model = MultiOutputRegressor(xgb)
206
207     scores = cross_val_score(
208         model, X, y, scoring=scorer, cv=CustomCV(folds, df["cluster"])
209     )
210     return scores.mean()
211
212
213 study = optuna.create_study(
214     direction="maximize", sampler=optuna.samplers.RandomSampler(seed=23)
215 )
216 # study = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(
217     seed=23))
218
219 study.optimize(objective, n_trials=4000, n_jobs=5)
220 print(f"\nBest Parameters:\n{study.best_params}\n")
221
222 os.makedirs("optuna/plots", exist_ok=True)
223 os.makedirs("optuna/trial-data", exist_ok=True)
224
225 timestamp = datetime.now().strftime("%d-%m-%Y%H-%M-%S")
226 study.trials_dataframe().to_csv(f"optuna/trial-data/optuna_trials_{timestamp}.csv")
227
228 fig_hist = optuna.visualization.plot_optimization_history(study)
229 fig_hist.write_html(f"optuna/plots/optimization_history_{timestamp}.html")
230
231 fig_imp = optuna.visualization.plot_param_importances(study)
232 fig_imp.write_html(f"optuna/plots/param_importances_{timestamp}.html")
233
234 fig_slice = optuna.visualization.plot_slice(
235     study, params=list(study.best_trial.params.keys())
236 )
237 fig_slice.write_html(f"optuna/plots/param_slice_{timestamp}.html")
238
239 open_webbrowser = True
240 if open_webbrowser:
241     webbrowser.open(
242         Path(f"optuna/plots/optimization_history_{timestamp}.html").resolve().as_uri()
243     )
244     webbrowser.open(
245         Path(f"optuna/plots/param_importances_{timestamp}.html").resolve().as_uri()
246     )
247     webbrowser.open(
248         Path(f"optuna/plots/param_slice_{timestamp}.html").resolve().as_uri()
249     )

```

6 do_experiment_v2.py

Purpose: Analyzes experiment results, generates feature importance and visualizations.

```
1
2 import polars as pl
3 import pickle
4 from sklearn.multioutput import MultiOutputRegressor
5 import matplotlib.pyplot as plt
6 from sklearn.model_selection import cross_val_score
7 from sklearn.metrics import make_scorer
8
9 from xgboost import XGBRegressor
10 from cell_util.ml import CustomCV
11 from calculations_util.distance import haversine_np
12
13 df = (
14     pl.read_parquet("data/parquet/processed/dl.with_cluster_cell_info.parquet")
15     .drop_nulls(["Longitude", "Latitude"])
16     .select(
17         "pl_distance",
18         "log1p_pl_distance",
19         "pl_per_meter",
20         "pl_predicted_longitude",
21         "pl_predicted_latitude",
22         "throughput_per_meter",
23         "spectral_efficiency",
24         "sinr_per_meter",
25         "sinr_over_rsrp",
26         "signal_loss_ratio",
27         "rsrq_delta",
28         "rsrp_delta",
29         "scaled_loss_ratio",
30         "mimo_throughput_potential",
31         "mimo_distance_ratio",
32         "sinr_per_antenna",
33         "efficiency_per_antenna",
34         "modulation_efficiency",
35         "mimo_nn0_cell",
36         "mimo_nn1_cell",
37         "mimo_nn2_cell",
38         "mimo_nn3_cell",
39         "Longitude",
40         "Latitude",
41         "NR_UE_Pathloss_DL_0",
42         "Longitude_nn0_cell",
43         "Latitude_nn0_cell",
44         "Longitude_nn1_cell",
45         "Latitude_nn1_cell",
46         "Longitude_nn2_cell",
47         "Latitude_nn2_cell",
48         "Longitude_nn3_cell",
49         "Latitude_nn3_cell",
50         "Azimuth_nn0_cell_cos",
51         "Azimuth_nn0_cell_sin",
52         "Azimuth_nn1_cell_cos",
53         "Azimuth_nn1_cell_sin",
54         "Azimuth_nn2_cell_cos",
55         "Azimuth_nn2_cell_sin",
56         "Azimuth_nn3_cell_cos",
57         "Azimuth_nn3_cell_sin",
58         "directional_rsrp_nn0_lon",
59         "directional_rsrp_nn0_lat",
60         "directional_rsrp_nn1_lon",
61         "directional_rsrp_nn1_lat",
62         "directional_rsrp_nn2_lon",
63         "directional_rsrp_nn2_lat",
64         "directional_rsrp_nn3_lon",
65         "directional_rsrp_nn3_lat",
66         "cluster",
67         "NR_UE_RSRP_0",
68         "NR_UE_Nbr_RSRP_0",
```

```

69         "NR_UE_Nbr_RSRP_1",
70         "NR_UE_Nbr_RSRP_2",
71         "NR_UE_Nbr_RSRP_3",
72         "NR_UE_SINR_0",
73         "NR_UE_RB_Num_DL_0",
74         "NR_UE_RSRQ_0",
75         "NR_UE_Nbr_RSRQ_0",
76         "NR_UE_Nbr_RSRQ_1",
77         "NR_UE_Nbr_RSRQ_2",
78         "NR_UE_Nbr_RSRQ_3",
79         "NR_UE_BLER_DL_0",
80         "NR_UE_Timing_Advance",
81         "rach_attempt",
82         "rach_ok",
83         "ho_attempt",
84         "ho_ok",
85         "App_Throughput_DL",
86         "NR_UE_Throughput_PDCP_DL",
87         "NR_UE_Ack_As_Nack_DL_0",
88         "NR_UE_NACK_Rate_DL_0",
89         "NR_UE_Power_Tx_PUSCH_0",
90         "NR_UE_Power_Tx_PRACH_0",
91         "NR_UE_NACK_Rate_UL_0",
92         "modulation_avg_encoded",
93         "modulation_avg_log2_bits",
94         "cce_level_encoded",
95         "mcs_index",
96         "centroid_lon_nn0_cell_coverage",
97         "centroid_lat_nn0_cell_coverage",
98         "centroid_lon_nn1_cell_coverage",
99         "centroid_lat_nn1_cell_coverage",
100        "centroid_lon_nn2_cell_coverage",
101        "centroid_lat_nn2_cell_coverage",
102        "centroid_lon_nn3_cell_coverage",
103        "centroid_lat_nn3_cell_coverage",
104        "intersection_2_centroid_lon",
105        "intersection_2_centroid_lat",
106    )
107    .to_pandas()
108 )
109
110 with open("data/folds/folds.pkl", "rb") as f:
111     folds = pickle.load(f)
112
113 params = {
114     "n_estimators": 212,
115     "learning_rate": 0.002587,
116     "max_depth": 9,
117     "reg_alpha": 0.004842,
118     "reg_lambda": 0.092771,
119     "subsample": 0.807007,
120     "colsample_bytree": 0.811025,
121     "min_child_weight": 1,
122     "grow_policy": "depthwise",
123     "max_leaves": 0,
124 }
125
126
127 reg = MultiOutputRegressor(
128     XGBRegressor(
129         **params,
130         sampling_method="uniform",
131         tree_method="hist",
132         n_jobs=-1,
133         objective="reg:squarederror",
134         seed=23,
135     )
136 )
137
138 y = df[["Latitude", "Longitude"]].to_numpy()
139
140 X = df.drop(["Latitude", "Longitude", "cluster"], axis=1).to_numpy()
141

```

```

142 scorer = make_scorer(score_func=haversine_np, greater_is_better=False)
143
144 scores = cross_val_score(reg, X, y, scoring=scorer, cv=CustomCV(folds, df["cluster"]))
145
146 print(f"Mean={scores.mean()}| Stddev = {scores.std()}\n")
147
148 print("Scores:\n", scores)
149
150 reg.fit(X, y)
151
152 feature_names = df.drop(["Latitude", "Longitude", "cluster"], axis=1).columns
153 for i, target_name in enumerate(["Latitude", "Longitude"]):
154     model = reg.estimators_[i]
155     importances = model.feature_importances_
156
157     importance_df = pl.DataFrame(
158         {"Feature": feature_names, "Importance": importances}
159     ).sort("Importance", descending=True)
160
161     print(f"\nTop 10 Feature Importance for {target_name}")
162     print(importance_df.head(10))
163
164
165 y_pred = reg.predict(X)
166
167 plt.figure(figsize=(8, 6))
168 plt.scatter(y[:, 1], y[:, 0], c="blue", label="True", alpha=0.5, s=10)
169 plt.scatter(y_pred[:, 1], y_pred[:, 0], c="red", label="Predicted", alpha=0.5, s=10)
170 plt.xlabel("Longitude")
171 plt.ylabel("Latitude")
172 plt.legend()
173 plt.title("True vs Predicted Locations")
174 plt.grid(True)
175 plt.tight_layout()
176 plt.show()

```

7 map_app.py

Purpose: Streamlit app to visualize coverage areas and data points spatially.

```

1
2 # Streamlit app displays a map of selected 5G cell coverages and user locations at ITU
   campus, allowing users to filter data by cell ID and neighbor range
3 import folium
4 import streamlit as st
5 from streamlit_folium import st_folium
6 import geopandas as gpd
7 import polars as pl
8
9 coverages = gpd.read_parquet("data/parquet/coverages/coverages.parquet")
10 df = pl.read_parquet("data/parquet/processed/dl.with_cluster.parquet").drop_nulls(
11     ["Longitude", "Latitude"]
12 ) # null columns prevents marking user equipment locations on the map
13
14 gdf = gpd.GeoDataFrame(
15     df.to_pandas(),
16     geometry=gpd.points_from_xy(df["Longitude"], df["Latitude"]),
17     crs="EPSG:4326",
18     columns=df.columns,
19 )
20
21 st.set_page_config(layout="wide")
22 st.title("ITU Coverage Map")
23
24 col_1, col_2 = st.columns([5, 2]) # width ratio of columns on the app
25
26 m = folium.Map(location=[41.10427, 29.02554], zoom_start=15) # created starting location
   of the map manually
27
28 columns_to_check = [

```

```

29     "NR_UE_PCI_0",
30     "NR_UE_Nbr_PCI_0",
31     "NR_UE_Nbr_PCI_1",
32     "NR_UE_Nbr_PCI_2",
33     "NR_UE_Nbr_PCI_3",
34 ]
35
36 with col_2:
37     selected_pci_list = st.multiselect("Choose Cell Id:", coverages["PCI"])
38
39     start_n, end_n = st.select_slider(
40         "Select the range of neighbor cells to include",
41         options=[0, 1, 2, 3, 4],
42         value=(0, 0),
43     )
44
45 coverages = coverages[coverages["PCI"].isin(selected_pci_list)]
46
47 mask = False
48
49 for col in columns_to_check[start_n : end_n + 1]:
50     mask |= gdf[col].isin(selected_pci_list)
51
52 gdf = gdf[mask]
53
54 folium.GeoJson(
55     coverages.clipped_coverage,
56     style_function=lambda x: {
57         "fillColor": "red",
58         "color": "black",
59         "weight": 1,
60         "fillOpacity": 0.2,
61     },
62 ).add_to(m)
63
64 folium.GeoJson(
65     gdf,
66     marker=folium.Circle(
67         radius=10,
68         fill=True,
69         fill_color="blue",
70         fill_opacity=0.2,
71         color="black",
72         weight=1,
73     ),
74 ).add_to(m)
75
76 with col_1:
77     st_folium(m, height=500, use_container_width=True)

```

8 coverage_intersections.py

Purpose: Calculates coverage areas and their intersections.

```

1
2 from itertools import combinations
3 from shapely.ops import unary_union
4 import geopandas as gpd
5
6 coverages = gpd.read_parquet("data/parquet/coverages/coverages.parquet")
7 coverages["clipped_coverage"] = coverages.clipped_coverage.to_crs("EPSG:32635")
8
9 intersections_2 = []
10
11 for cell_1, cell_2 in combinations(coverages["PCI"], 2):
12     coverage_poly_1 = coverages[coverages["PCI"] == cell_1].clipped_coverage.iloc[0]
13     coverage_poly_2 = coverages[coverages["PCI"] == cell_2].clipped_coverage.iloc[0]
14
15     intersection = coverage_poly_1.intersection(coverage_poly_2)
16

```



```

17     if intersection.is_empty:
18         convex_hull_flag = 1
19         combined = unary_union([coverage_poly_1, coverage_poly_2])
20         convex_hull = combined.convex_hull
21         area = convex_hull.area
22         shape = convex_hull
23
24     else:
25         convex_hull_flag = 0
26         area = intersection.area
27         shape = intersection
28
29     intersections_2.append(
30         {
31             "sorted_cell_pairs": sorted([cell_1, cell_2]),
32             "convex_hull": convex_hull_flag,
33             "area_m2": area,
34             "geometry": shape,
35         }
36     )
37
38 intersections_2 = gpd.GeoDataFrame(intersections_2, geometry="geometry", crs="EPSG:32635")
39 intersections_2["centroid"] = intersections_2.geometry.centroid.to_crs("EPSG:4326")
40 intersections_2 = intersections_2.to_crs("EPSG:4326")
41
42 intersections_2.to_parquet("data/parquet/coverages/coverages_intersect_2.parquet")

```